UNIVERSITÀ
DI PARMA

Dipartimento di Scienze Matematiche, Fisiche e Informatiche

Corso di Laurea Triennale in Informatica

Modellazione del Comportamento Dinamico nella Robotica Sociale: Sincronizzazione della Cinematica Motoria e della Logica Basata sulle Emozioni per la Piattaforma NAO

Dynamic Behavior Modeling in Social Robotics: Synchronizing Motor Kinematics and Emotion-Driven Logic for the NAO Platform

Candidato:
Khulekani Hillary Mthethwa

Relatore:
Prof. Alessandro Dal Palù

Correlatore:
Prof. Riccardo Monica

Anno Accademico 2025–2026

Sommario

La tesi illustra il progetto svolto durante i tirocini per l'implementazione di gesti e movimenti sviluppati per il robot NAO, al fine di migliorarne le prestazioni come Socially Assistive Robot (SAR). I sistemi SAR forniscono supporto ai pazienti, aiutando a ridurre il carico di lavoro dei professionisti sanitari. Il progetto preesistente presso l'Università di Parma presentava specifiche limitazioni, tra cui un'esecuzione statica dei dialoghi, movimenti limitati e un'assenza di sincronizzazione tra parola e movimento.

Per superare tali criticità e trasformare il robot in un compagno empatico e coinvolgente, sono state implementate diverse modifiche e migliorie. I contributi principali includono lo sviluppo di una libreria di movimenti e gesti personalizzata, concepita per evitare collisioni utilizzando l'ambiente Choregraphe e l'apprendimento cinestetico. A ciò si aggiunge l'esportazione matematica delle animazioni, basate sull'interpolazione di Bézier, in array di esecuzione Python, garantendo così un'integrazione autonoma tramite il framework ALMotion.

Il lavoro introduce un innovativo strumento di sincronizzazione multithread, il quale impiega un buffer di avvio hardware (spin-up buffer) e lock basati su callback a singola esecuzione (single-fire) per allineare fluidamente il parlato sintetico all'esecuzione motoria. Questo strumento è integrato da movimenti o gesti dinamici basati sulle emozioni dell'utente rilevate in tempo reale.

Le interazioni simili a quelle umane sono preservate tramite una regola di sparsità comportamentale 30/70, incaricata di modulare la reazione del NAO agli stimoli emotivi. Parallelamente, il motore logico ASP (Answer Set Programming) è stato esteso in modo da integrare la generazione del parlato e la mappatura delle azioni fisiche all'interno di un unico processo decisionale.

La validazione, effettuata sulla piattaforma fisica del NAO presso il Robotics and Intelligent Machines Laboratory (RIMLab) dell'Ateneo parmense, ha confermato l'efficacia del framework. Durante le attività, il sistema ha dimostrato una corretta esecuzione dei gesti, una perfetta sincronizzazione tra movimenti e audio, nonché una solida stabilità. Queste modifiche, realizzate

per la modellazione del comportamento dinamico, garantiscono che il robot NAO interagisca con i pazienti in modo naturale, con espressività ed empatia.

Contents

Sommario **3**

1 Introduction **3**
1.1 Background on Socially Assistive Robots and Cognitive Rehabilitation . . . 3
1.1.1 The Need for Cognitive Rehabilitation . . . 4
1.1.2 Challenges of Traditional Cognitive Rehabilitation . . 4
1.1.3 Socially Assistive Robots . . . 4
1.1.4 Socially Assistive Robots in Cognitive Rehabilitation . 5
1.2 The NAO Humanoid Robot . . . 6
1.2.1 Overview of the NAO Platform . . . 6
1.2.2 Hardware Architecture . . . 7
1.2.3 Sensors and Actuators . . . 8
1.2.4 Software Framework . . . 10
1.2.5 Strengths and Limitations . . . 11
1.3 Research Motivation . . . 14
1.4 Thesis Objectives and Contributions . . . 15

2 State of the Art **17**
2.1 Introduction . . . 17
2.2 The University of Parma Cognitive Rehabilitation Project . . 17
2.2.1 Project Objectives . . . 17
2.2.2 Rehabilitation Workflow . . . 18
2.2.3 Existing System Architecture . . . 18
2.2.4 Existing Technologies Used . . . 19
2.3 Existing Human–Robot Interaction . . . 19
2.3.1 Dialogue Management . . . 19
2.3.2 Emotion Recognition . . . 20
2.3.3 Answer Set Programming (ASP) . . . 20
2.3.4 Session Flow . . . 20
2.4 Limitations of the Existing System . . . 21

2.5 Chapter Summary 21

3 Technologies and Methodologies 23
3.1 Introduction 23
3.2 NAO Motion Control 24
3.2.1 ALMotion 24
3.2.2 Joint-Based Motion and Keyframes 24
3.2.3 Bezier Interpolation 24
3.3 Gesture Development with Choregraphe 25
3.3.1 Animation Timeline 25
3.3.2 Virtual Prototyping 25
3.3.3 Kinesthetic Teaching 25
3.4 Speech Synthesis and Audio Processing 25
3.4.1 Text-to-Speech 26
3.4.2 Edge-TTS 26
3.4.3 Audio Playback on NAO 26
3.5 Emotion Recognition 27
3.6 Answer Set Programming (ASP) 27
3.6.1 ASP Fundamentals 27
3.6.2 Behaviour and Decision Logic 28
3.6.3 feedback.asp 28
3.7 Multithreading and Synchronization 28
3.8 Chapter Summary 29

4 Dynamic behaviour modelling 31
4.1 Introduction 31
4.2 Custom Gesture Engineering 32
4.2.1 Gesture Design Requirements 32
4.2.2 Choregraphe-Based Gesture Development 32
4.2.3 Kinesthetic Teaching 32
4.2.4 Bezier Keyframe Export and Python Integration . . . 33
4.2.5 Gesture Execution through ALMotion 33
4.3 Multimodal Speech–Motor Synchronization 33
4.3.1 Existing Synchronization Problem 33
4.3.2 Audio–Motor Latency 34
4.3.3 Thread Blocking and Pre-emption 34
4.3.4 Hardware Spin-up Buffer 34
4.3.5 Motor Command Collision 36
4.3.6 Callback Management and Edge-TTS 36
4.3.7 Final Synchronization Pipeline 36
4.4 Dynamic Emotion-Driven Behaviour 37

4.4.1 Dynamic Emotion-Driven Behaviour 37
4.4.2 Dynamic Helper Functions 37
4.4.3 Context-Aware Physical Responses 37
4.4.4 Behavioural Sparsity and the 30/70 Rule 37
4.5 ASP Behaviour Integration 38
4.5.1 Existing Behaviour Logic 38
4.5.2 Extension of feedback.asp 38
4.5.3 Behaviour Facts 38
4.5.4 Integration with Dialogue States 40
4.5.5 Dynamic Behaviour Selection 40
4.6 Overall System Architecture 40
4.7 Chapter Summary 41

5 Validation and Evaluation 43
5.1 Introduction 43
5.2 Experimental Setup 43
5.2.1 NAO Hardware Configuration 43
5.2.2 Software Environment 43
5.2.3 Test Scenarios 44
5.3 Gesture Validation 44
5.3.1 Hardware Safety Protocol and Instability 44
5.3.2 Validation of Kinesthetic Teaching 45
5.3.3 Gesture Library Deployment 45
5.4 Multimodal Synchronization Evaluation 47
5.4.1 Audio–Motor Timing 47
5.4.2 Thread and Callback Stability 47
5.5 Emotion-Driven Behaviour Evaluation 47
5.5.1 Offline Simulation and Logic Verification 47
5.5.2 Behavioural Sparsity Evaluation 47
5.6 Integrated System Validation 47
5.6.1 Live Code Execution and Log Monitoring 47
5.6.2 Iterative Debugging and Error Resolution 48
5.6.3 System Stability and Excel Logging 48
5.7 Discussion 48
5.8 Chapter Summary 48

6 Conclusion and Future Work 49
6.1 Introduction 49
6.2 Summary of the Work 49
6.3 Main Contributions 50
6.4 Limitations 50

6.5 Future Work 51
6.6 Final Remarks 52

Acknowledgements **53**

Bibliography **55**

List of Figures

1.1 Different Socially Assistive Robots [12] 3
1.2 Socially Assistive Robot [13] 5
1.3 NAO Robot [14] 6
1.4 NAO Hardware [15] 7
1.5 NAO's primary visual and auditory sensors [16] 8
1.6 Choregraphe environment [17] 11
1.7 Deadlock [18] 13
1.8 Patient interacts with NAO [19] 15

2.1 Trio of NAO, patient and therapist [20] 18
2.2 Overview of the existing architecture [21] 19

3.1 NAO and NAOqi system [22] 23
3.2 NAO Keyframes and timestamps [23] 24
3.3 Speech processing [24] 26
3.4 ASP model [25] 27
3.5 Threading illustration [26] 28

4.1 Choregraphe interface [27] 32
4.2 NAO being handled [28] 33
4.3 A 0.4s motor delay, that can be tweaked 34
4.4 Adding the delay 35
4.5 Changed order to give the motors a head start 35
4.6 Changed order to give the motors a head start 35
4.7 Swapped the order and put the callback before the remote play 35
4.8 Single-fire callback. 36
4.9 Part in say_with_me block 36
4.10 Helper function that automates gestures based on detected emotions 37
4.11 Sparsity rule 38
4.12 Part of feedback.asp extension 38
4.13 Gestures mapped to specific behaves 39

4.14 Feedback expansion 39
4.15 Feedback expansion 39
4.16 Updated say_with_me 40
4.17 Reason was crashing because of missing variables 40
4.18 Overview of the system architecture 41

5.1 Box libraries panel [29] 46

Chapter 1

Introduction

1.1 Background on Socially Assistive Robots and Cognitive Rehabilitation

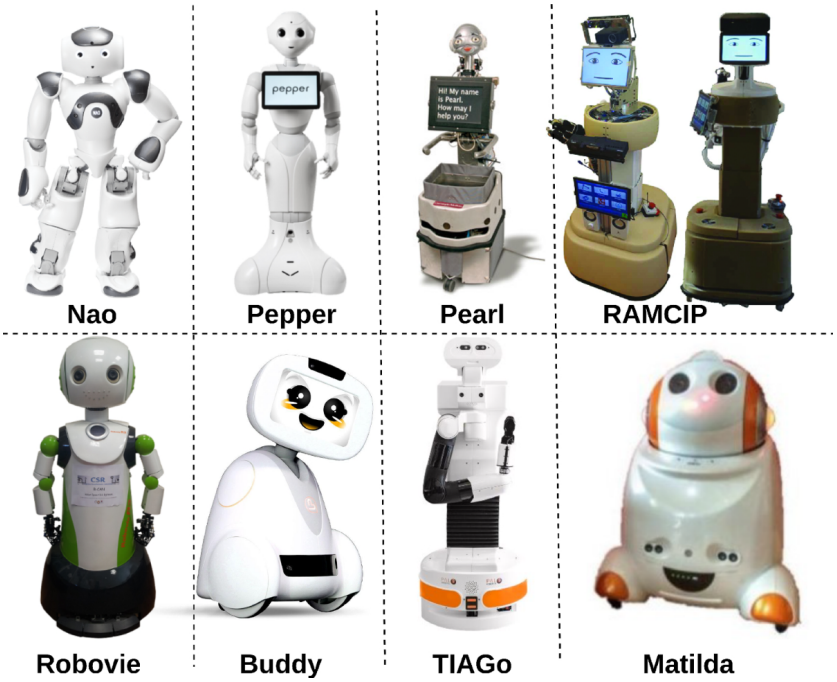


Figure 1.1: Different Socially Assistive Robots [12]

1.1.1 The Need for Cognitive Rehabilitation

In modern times, a fast growing aging population is one of the biggest issues experienced in the healthcare sector. Due to safer living conditions, improved nutrition, a lower child mortality rate, and improvements in the medical science sector, the world life expectancy continues to increase. While this is a positive development, it has also resulted in the increase of the amount of neurological disorders, age related cognitive decline and brain injuries [1].

Conditions such as stroke, Parkinson's disease, Alzheimer's disease, and Mild Cognitive Impairment (MCI) disproportionately affect the elderly. These conditions often affect one's memory, attention, problem solving abilities, and other cognitive functions, reducing an individual's independence to do things without any assistance and overall quality of life. As a result, there has been a significant increase in the demand for effective cognitive rehabilitation actions [1], [2].

The primary goal of cognitive rehabilitation is to improve or recover some declining cognitive functions through organized therapeutic activities designed to improve one's cognitive performance [2]. Different rehabilitation methods have been developed to help patients with their slow down or recover cognitive decline.

1.1.2 Challenges of Traditional Cognitive Rehabilitation

Cognitive rehabilitation is still facing its own challenges. Because of a shortage of healthcare professionals in some countries it is difficult to provide continuous and personalized rehabilitation services to every patient. The increase in cost of healthcare and the long-term nature of rehabilitation programmes place significant pressure on healthcare systems worldwide [1].

These limitations have encouraged researchers to investigate innovative and technology-assisted approaches capable of supporting healthcare professionals while improving the accessibility and quality of rehabilitation services.

1.1.3 Socially Assistive Robots

Thanks to the rapid growth of Artificial Intelligence significant advances have been made in developing intelligent technologies for healthcare. One of the most promising innovations is the use of Socially Assistive Robots (SARs), which have become an effective tool for cognitive rehabilitation.

Socially Assistive Robots took off around the 2000s with the main aim of assisting patients through social engagement and support rather than phys-

ical interaction [3]. Unlike traditional assistive robots, SARs communicate with users through speech, gestures, facial expressions, eye contact and other social behaviours. Their engagements with patients are designed to motivate, guide, instruct, encourage, and provide companionship during therapeutic sessions [3].

One of the main advantages of these socially assistive robots is their ability to personalize interactions according to each user's needs. Emulating the communication style, behaviour, and mannerisms of a user or patient these robots can produce more engaging and meaningful experiences, improving patient motivation and encouraging the patient to participate.

1.1.4 Socially Assistive Robots in Cognitive Rehabilitation

The application of Socially Assistive Robots cognitive rehabilitation has been the topic for a lot of researchers over the past two decades. They have an ability to combine cognitive training with natural social interaction that allows them to complement conventional rehabilitation by providing personalized assistance, continuous encouragement, and giving real-time feedback throughout therapy sessions [4].



Figure 1.2: Socially Assistive Robot [13]

Among different socially assistive robotic platforms developed for healthcare applications, the NAO humanoid robot has become one of the most utilised platforms in research due to its expressive capabilities, rich sensory

system, and flexible programming environment. The next section provides an overview of the NAO platform, its hardware architecture, sensing capabilities, software framework and existing limitations, which form the foundation for the work presented in this thesis.

1.2 The NAO Humanoid Robot



Figure 1.3: NAO Robot [14]

1.2.1 Overview of the NAO Platform

The NAO robot is a Socially Assistive Robot (SAR) and a social companion humanoid robot created by the French company Aldebaran. The NAO robot is designed for human interaction, healthcare, and research with a main focus on non-physical assistance [5]. NAO generally carries out assistance such as tutoring, assisting the elderly, children with autism and patients in old-people's homes or hospitals.

Researchers prefer the NAO Robot because of its affordability, versatility, friendly aesthetic, and safety. The NAO has a non-threatening design, it is lightweight, possesses round edges and has a child friendly appearance. The NAO is portable which makes it fairly easy for researchers, students and users to carry or move it around, this also allows the ease of deployment. Another attractive characteristic of the NAO Robot is its vast Developer Ecosystem which allows Python SDKs, NAOqi and ROS integration.

The versatility of the NAO is a major reason why it is used across multiple factors such as:

- **Healthcare & Therapy:** Tasks for cognitive rehabilitation, autism therapy, assistance games
- **Computer Science & AI research:** Effective tool for computer vision testing, motion planning, improving a robot's walking, generation of dynamic gestures and emotion recognition.
- **Education:** Acts as an interactive tutor for different subjects like language, mathematics, basic elementary subjects etc

1.2.2 Hardware Architecture

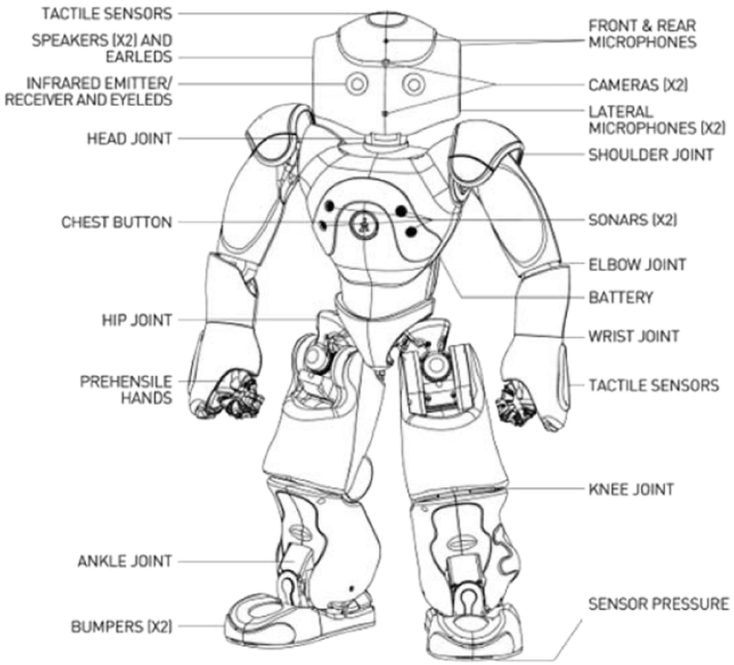


Figure 1.4: NAO Hardware [15]

The NAO Robot stands at approximately 58 cm tall and weighs 5.5 kg. NAO utilises a distributed hardware architecture powered by a main CPU board in the head and a secondary chest board with an ARM processor which increases efficiency by executing the simpler instructions [6]. This results in lower power consumption and prevents overheating. NAO is a humanoid featuring a total of 25 degrees of freedom, allowing it to carry out complex dynamic movements. The DoF are spread as follows:

- **Head (2 DoF):** Controls both the yaw and the pitch, enabling the robot to look around, track objects, and make expressive head movements.
- **Arms (10 DoF total):** Each arm has 5 DoF, allowing a wide range of motion for gesturing, stretching, pointing, and reaching.
- **Hands:** Each hand has one degree of freedom.
- **Pelvis:** Controls the hips for maintaining the robot's stability and coordinating movements.
- **Legs:** Each leg has five degrees of freedom, allowing walking, standing, sitting and dynamic tasks like dancing or kicking.

The NAO Robot's eyes are equipped with multi-color LEDs, this allows the change of colors and patterns to boost NAO's expressiveness of emotions and indicate current state. This also gives the robot the ability of having more natural interactions with the users.

The legs of the NAO are the most robust part of the robot built to support its 5.48 kg weight and carry out complex bipedal movement, balancing and dynamic tasks like kicking or dancing.

The arms contain motorized revolute joints. They are built with coreless DC motors and position sensors to control complex, human-like upper-body movements.

1.2.3 Sensors and Actuators

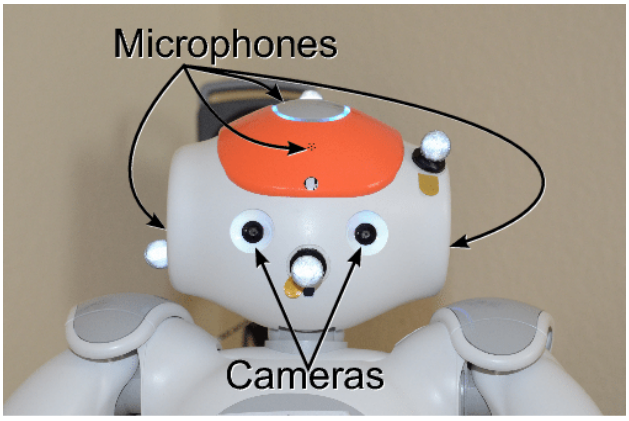


Figure 1.5: NAO's primary visual and auditory sensors [16]

Cameras: NAO is equipped with two HD cameras including one positioned at the forehead and one at the mouth level. The cameras handle the tasks requiring computer vision such as facial recognition, object tracking and shape detection, providing the main visual input for human-robot interaction [6].

Microphones: As shown in Figure 1.4, NAO has four microphones distributed around its head. This set of microphones allow spatial sound localization, giving NAO the ability to identify the direction in which the sound is coming from (source) and turn its head toward a speaker during its interaction with users.

Speakers: The NAO features two high-fidelity loudspeakers located on the sides of the head or the ears. The speakers are used for playing audio files, auditory feedback and executing text-to-speech outputs during engagements or interactions [6].

Touch Sensors: To support direct physical interactions, capacitive tactile sensors are embedded in the NAO Robot's hardware. These sensors are located on the top of the head, divided into three distinct zones and on the backs of the hands, allowing NAO to respond to physical cues like touch or being patted [6].

Sonar: These are two ultrasonic emitters and receivers which are housed in NAO's chest panel. These sensors measure the flight time of sound waves to calculate the distances of the surrounding obstacles, this is very critical for collision avoidance and safer navigation [6].

Inertial Sensors: The stability and posture of the NAO are maintained through an Inertial Measurement Unit (IMU) placed in the centre of the torso. Features a 2-axis gyrometer and a 3-axis accelerometer, the IMU continuously computes NAO's centre of mass and orientation to prevent falls and maintain equilibrium [6].

Kinematics: The Actuator Network Accurate actuators that convert computational commands into physical motion drive the movement in the NAO robot.

- **Joint Motors:** Maxon coreless brush DC motors coupled with precision gearboxes, monitored by Magnetic Rotary Encoders (MREs) that provide angular positional feedback in real-time to the software in control, ensuring high precision movement.

- **Head Movement:** The neck is motorized by a 2 Degree of Freedom (DoF) joint controlling yaw and pitch, enabling the robot to orient its cameras and microphones towards the targets of interest.

- **Arm Movement:** For Generating smooth, fluid, expressive upper

body gestures, usually used for tasks, interactions and non-verbal communication the NAO's is equipped with mobile upper limbs featuring 5 DoF per arm (shoulder pitch/roll, elbow yaw/roll, and wrist yaw).

- **Hand Opening/Closing:** Each hand contains a motor that drives the NAO's prehensile fingers. While featuring one DoF per hand [6].

- **Walking:** The bipedal architecture is designed for dynamic walking, balancing and weight shifting. The NAO was designed to mostly focus on communicative gesturing so the upper body does most of the work with the lower body actuators largely used to maintain stability and static posture [6].

1.2.4 Software Framework

A software ecosystem makes the physical capabilities and core functions of the NAO robot manageable and controllable. Resource sharing, multitasking and easy communication between the NAO's movements, speech and visual systems are all enabled by the software framework. In addition, developers can program custom behaviours and carry out complex kinematic and iterative routines.

Core Software Components

- **NAOqi Middleware:** NAOqi is the primary software that runs on the NAO Robot and other robots created by Aldebaran such as the Pepper and Romeo. NAOqi provides a distributed architecture that runs as a cross-platform, cross-language middleware that deals with inter-process communication, hardware decoupling or device abstraction and event management such as scheduling [6].

- **Choregraphe:** A visual programming and simulation environment used for designing custom animations, movements, gestures and behaviours [7]. It utilizes a timeline based interface for keyframe animation. Choregraphe could be a useful tool even without the physical robot because it allows developers to work with a virtual robot. The choreography platform supports real-time synchronisation with the hardware, enabling the physical NAO to concurrently mirror the movements or gestures being executed within the virtual environment.

- **Python SDK:** The NAOqi Python SDK allows developers to interact and communicate with the robot's operating system. An ALProxy can be created to specific NAOqi modules, allowing developers to write

their own Python scripts to dynamically control the robot over a network connection, sidestepping the limitations of purely visual programming. It is one of the most flexible frameworks [6].

ALProxy is an object that acts as a bridge for a specific module on the NAO. It can call functions instead of the developer having to write complex network code.

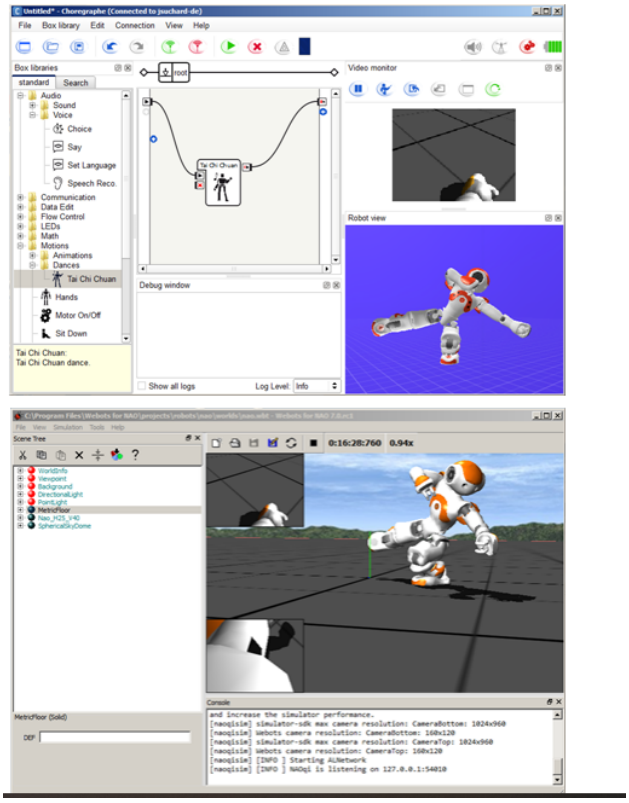


Figure 1.6: Choregraphe environment [17]

1.2.5 Strengths and Limitations

Understanding the NAO Robot as a deployment platform means understanding its strengths and limitations. Knowing areas NAO excels in and areas it falls short in is helpful in deciding whether to use the NAO or to use different alternatives.

Strengths

- **Human-like appearance:** The NAO humanoid is built with an aesthetically pleasing appearance that makes it non-threatening, friendly and approachable. The NAO is built to encourage interaction or engagement with everyone from all age groups.

- **Expressiveness:** The robot is capable of executing a variety of gestures, it is able to stand in different postures and execute a number of body language cues. One of the biggest advantages of the NAO is its ability to communicate without using any verbal actions. This adds on to the long list of why the NAO is preferred by researchers and in healthcare.

- **Easy to program:** Having the Choregraphe interface and the Python SDK makes it a lot easier for researchers, programmers and learners to prototype and implement custom behaviour. This can be useful for educational purposes as it gives users different options to experiment with.

- **Extensive sensor array:** The NAO Robot boasts a robust, multimodal platform for the environment perception and its interaction with users because it is equipped with tactile sensors, cameras, sonar and microphones.

- **Well documented:** The NAO has been active for a significant number of years which has allowed countless tests, updates and many improvements. All these make it possible to keep adding on to the existing documentation as well as use it for future documentation. There exists a vast repository of resources, guides and open-source libraries run by a community.

- **Wisely used:** Due to the NAO's popularity and it being common in human-robot interaction (HRI) research, it is much easier to improve, validate and compare results of different projects. This contributes significantly to the development and progression of HRI.

Limitations

- **Limited processing power:** The CPU has constraints in the number of tasks running concurrently. This limits the robot's ability to run the most modern artificial intelligence models eg vision deep learning, advanced natural language processing, it is not able to understand some languages.

- **Outdated hardware:** The NAO is a legacy platform, which means that even though hardware infrastructure is outdated, the companies continue to use it. Therefore its internal hardware components trail behind the modern day technological standards, restricting it in multiple areas including sensor resolution and computational bandwidth.

- **Limited gesture expressiveness:** Despite being able to perform broad movements, the default set can be repetitive and can lack real human like expressions which often requires users to create their own custom movements.

- **Threading issues:** Complex concurrent process management is an area of major concern for the NAOqi. The contention of resources between the vision, motion and audio threads can lead to a common process management issue called 'Deadlock'. A deadlock occurs when multiple processes are unable to continue because each of them is waiting for another to release a resource required by another.

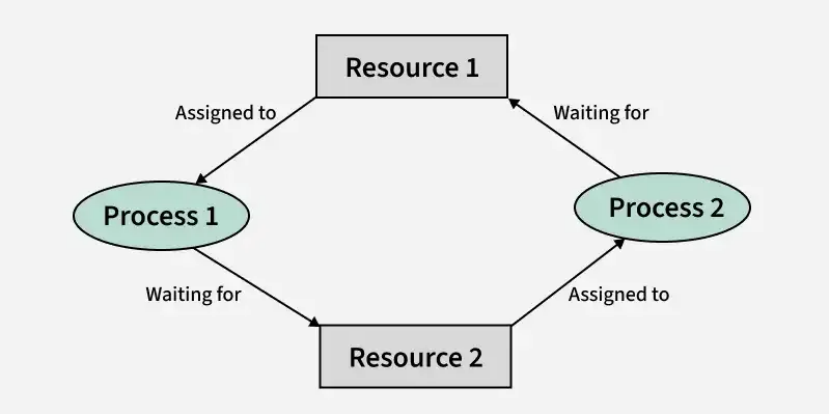


Figure 1.7: Deadlock [18]

- **Model interruptions:** The NAO Robot often stutters, jitters and sometimes has sudden stops while it's in motion. This is caused by the competing software running in the background and makes it tougher for the developers to create smooth movements.

- **Restricted natural conversation:** As of now it is very difficult to implement natural conversations with the NAO. It mostly relies mostly on scripted dialogue which can be very limited and unnatural.

Despite the strengths and the solid platform for socially assistive robotics provided by the NAO, the limitations exhibited by the NAO platform bring forth the need for further development of the dynamic behaviour modelling framework presented in this thesis.

1.3 Research Motivation

The University of Parma through its researchers, students, developers and in collaboration with other universities launched a groundbreaking research initiative aimed at the integration of Socially Assistive Robots into different environments such as healthcare to support nurses, therapists, patients, the elderly and different people across all sectors in life. All this is being done by utilising the NAO humanoid robot and its abilities. The early stages of this project were heavily focused on setting up the interactive architecture, with main emphasis on the development of a core conversational system or the dialogue, Answer Set Programming(ASP) which is used for logical reasons and dialogue management, in addition camera based facial recognition which is used to recognise emotions. A web-based management interface called the "Dashboard" is also implemented.

Despite all the progress and milestones achieved, a complete socially assistive robot would require other areas to be developed and integrated into the system, not just the sophisticated backend reasoning. The dialogue was mostly static, there was enough speech but a lack of action, it seemed unnatural, lacked substance and would not be able to encourage, motivate or excite users. This was a huge drawback because a robot's physical gestures are crucial communicative elements which cannot be ignored in socially assistive robotics. With the already existing limited gestures, the timing between the NAO Robot's speech or audio and the physical gestures were not synchronised. In addition to that the system was not reacting to the user's emotions accordingly. All this resulted in dialogue that lacked depth, interactions were repetitive, robotic, stilted and lacked human-like fluidity in its overall behaviour. It was simply not empathetic enough in terms of interaction feedback for a standard SAR.

Achieving short and long-term success in the development and use of SARs requires these limitations to be overcome. In the case of patients undergoing cognitive rehabilitation, the fluidity and natural communication of the NAO robot is essential for the human-robot interaction and an engagement that can bring the desired results such as improving the patient's health, making the patient feel better and making the HRI method a preferred method. It is much easier for a patient to want to interact with a

robot if it has the natural flow of a human and reacts quick enough before the patient loses interest. Sustaining interactions that mirror normal human reactions is what will make NAO a complete SAR.

This thesis addresses these limitations and closes the gap between the internal robot logic and its physical expressions such as gestures and movements by developing a dynamic behaviour modelling framework for the NAO Robot. The proposed framework successfully integrates the robot's speech, audio, motor kinematics, reactions, responses and decision-making logic. The existing work is improved and polished by resolving these current existing issues. From a static dialogue system into a fully functional reactive, asynchronous and event-driven platform which will benefit patients in their rehabilitation sessions.

1.4 Thesis Objectives and Contributions

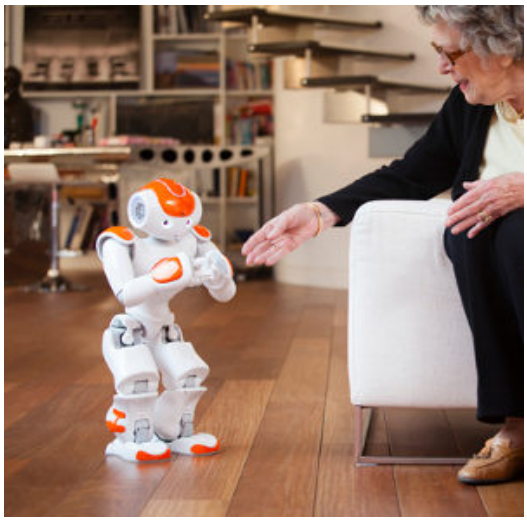


Figure 1.8: Patient interacts with NAO [19]

The primary goal of this thesis, originating directly from the work completed during the research internship, is to address the interactive limitations of the existing cognitive rehabilitation project carried out at the University of Parma. To enhance the effectiveness of how the NAO engages users, several improvements, changes, updates and modifications were designed and implemented during the internship project.

The primary contributions include:

- **Development of Custom Gestures:** Creation of a library with custom gestures using the Choregraphe environment. The movements were vastly tailored for interactions with patients and have the ability to make patients happy, laugh and improve their overall well being.
- **Dynamic Animation Integration:** Integration of existing gestures into different routines with speech. In addition the newly created movements and gestures were mathematically exported for standalone execution giving researchers and developers the ability to choose specific movements to work with.
- **Speech and Motor Synchronisation:** The existing speech was combined with gestures and movements with seamless synchronisation. The improvement resolved latency issues especially with the hardware, ensuring there is natural coordination between physical movements, gestures and the speech or audio.
- **Emotion-Driven Reactivity:** Using the facial recognition feature to detect the user's emotions e.g. happy, sad or surprised, in real time and triggering an appropriate response, action or gesture.
- **Conversational Behavioural Sparsity:** Implementation of a 30/70 function that decides which appropriate gestures to implement. The 30/70 rule balances between executing a simple and small gesture or a bigger appropriate action to ensure the robot is not just standing and it is not repetitive.
- **Enhanced Decision Engine Logic:** Refinement and expansion of the Answer Set Programming (ASP) logic engine to integrate movements and gestures alongside the dialogue. Specific physical actions were directly linked to specific dialogue.
- **System Validation and Tracking:** Validation of the modifications through offline simulation blocks and extensive testing on the physical NAO robot. This included updating interaction logging parameters to ensure system stability during active memory exercises.

Chapter 2

State of the Art

2.1 Introduction

This chapter maps out the foundational context of the research by evaluating the existing work and the system prior to the integration of the dynamic modelling framework. This thesis adds on to the work that was already done, building onto the ongoing projects and research. It is important to establish the boundary between the inherited project, existing architecture and the contributions in this work. The following sections break down the legacy architecture of the University of Parma's cognitive rehabilitation project, analyzing its objectives, workflow and foundational technologies. In addition this chapter reviews the already existing state of the HRI within the system, with main emphasis of managing the dialogue, recognising emotions and Answer Set Programming(ASP) logic, highlighting the drawbacks that inspired this research.

2.2 The University of Parma Cognitive Rehabilitation Project

2.2.1 Project Objectives

The overarching goal of the ongoing University of Parma projects is to provide an advanced, interactive cognitive rehabilitation tool to support healthcare professionals. The deployment of these SARs benefit elderly patients suffering from multiple health issues like memory loss and strokes. Introducing socially assistive robotics into clinical and therapeutic environments should be done when these robots are capable of having solid human-robot interactions which is something this project aims to do. In short the project aims

to reduce the overall workload on the medical stuff by providing the services of these SARs.

2.2.2 Rehabilitation Workflow

Usually the rehabilitation workflow involves a trio of the patient, the robot, and the supervising therapist. The therapist sets up the parameters before the session begins, selecting tailored cognitive exercises that tend to the patient's needs. During the session, the NAO robot autonomously guides the patient through these tailored exercises, drawing out responses, providing feedback, and progressing through the therapeutic modules while the therapist monitors the interaction and the patient's progress. There is collaboration between the NAO and the therapist.



Figure 2.1: Trio of NAO, patient and therapist [20]

2.2.3 Existing System Architecture

The already existing system architecture operates on a distributed model that decouples the physical NAO robot hardware from the intensive computational logic. A centralized backend server handles the processing of the high-level logic and reasoning, connected over a network to the NAO robot, which serves as the physical interface (the frontend) for the patient. The system has logs which track the performance and these can be used to analyse, study and even assess the cognitive progress.

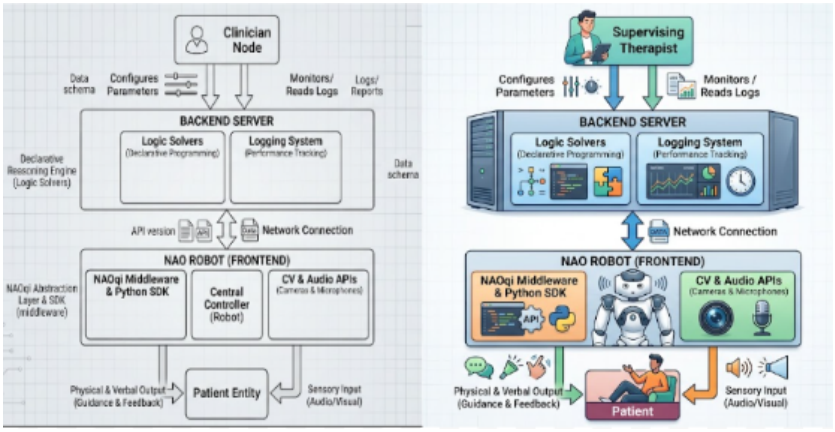


Figure 2.2: Overview of the existing architecture [21]

2.2.4 Existing Technologies Used

Different technologies are used and relied on to operate:

- **NAOqi & Python:** For basic hardware control, sensory data harvesting and network communication were handled by the NAO's middleware and Python SDK.
- **Logic Solvers:** The logic-solving mechanism relied extensively on declarative programming environments as the backbone for backend reasoning and decision-making, where logical rules were processed to determine the robot's subsequent behaviours.
- **Computer Vision/Audio APIs:** Pre-existing integrations handled the input from cameras and microphones to process and produce the outputs.

2.3 Existing Human–Robot Interaction

The centre of the rehabilitation project relies on how effectively the robot can communicate and interact with the patient. HRI components functioned as follows:

2.3.1 Dialogue Management

The pre-existing dialogue management system handled the conversational exchange between the NAO robot and the patient. It was mainly responsible for processing user intentions and selecting the most suitable textual

response. However, this dialogue execution was lacking fluidity, relying on predefined scripts and sequential conversational trees to guide the patient through the exercises.

2.3.2 Emotion Recognition

The focus on recognising the user's emotion was an important feature in the inherited project. The system utilized camera feeds and facial recognition algorithms to determine the patient's emotional state (e.g. sad, surprised, happy, frustrated, confused) during the session. This data was intended to inform the robot's behaviour, allowing it to react accordingly like offering encouragement if a patient appeared frustrated or positive reinforcement when they succeeded.

2.3.3 Answer Set Programming (ASP)

Answer Set Programming (ASP) is used to handle the logic and the complex decision-making for the dynamic therapy session. ASP is a form of declarative programming oriented towards difficult search problems [8]. In this project, ASP was used as the "brain" of the operation. By giving information to the ASP solver (e.g., the current stage of the exercise, the patient's mood detected, the history of responses), the system could logically calculate the most appropriate step to take or dialogue prompt for the robot to execute.

2.3.4 Session Flow

The typical session flow operated in a sequential loop:

1. The robot issues a prompt or exercise question.

2. The system waits for the patient's verbal response and visual emotional cues.

3. This multimodal data is passed to the ASP backend.

4. The ASP solver computes the next logical state.

5. The backend sends the resulting text string back to the robot for execution.

2.4 Limitations of the Existing System

While the solid foundational architecture successfully executed basic interactions and cognitive exercises, there remained several significant limitations in the robot's behavioural execution:

- **Static Dialogue Execution:** The conversation felt robotic and lacked the naturalness required by SARs. Because the dialogue was scripted it became very predictable and repetitive.
- **Limited Gesture Support:** The choice or the options for the gestures were very limited and often left the robot just speaking without any movements or with the same one or two movements over and over again. Interactions frequently consisted of speech devoid of any physical movement. Of the few gestures that were available, they simply did not have any correlation to the context of the conversation.
- **Lack of Speech-Motion Synchronization:** When some gestures were implemented, there was no synchronisation between the speech and the gestures. Either the physical movement or the speech would terminate prematurely, resulting in a disjointed interaction with no natural flow.
- **Limited Emotion-Driven behaviour:** Even though the robot could see and detect the emotions of the user, the hardware was not programmed to react accordingly.

Motivation for the Proposed Improvements: The gap between speech of the NAO and the physical movements is too great to ignore and closing down that gap is one of the primary focuses of this internship.

2.5 Chapter Summary

The chapter describes the state of the art regarding the University of Parma cognitive rehabilitation project before the thesis. It explains the system's goals, use of ASP for logic and reasoning, the abilities and capabilities of the HRI, including dialogue, interactions and the recognition of emotion. It identified the limitations such as the staticness of the dialogue, poor synchronization of limbs and speech that caused ineffective engagements with the NAO. The next chapters will detail the design and the implementation of the proposed solutions.

Chapter 3

Technologies and Methodologies

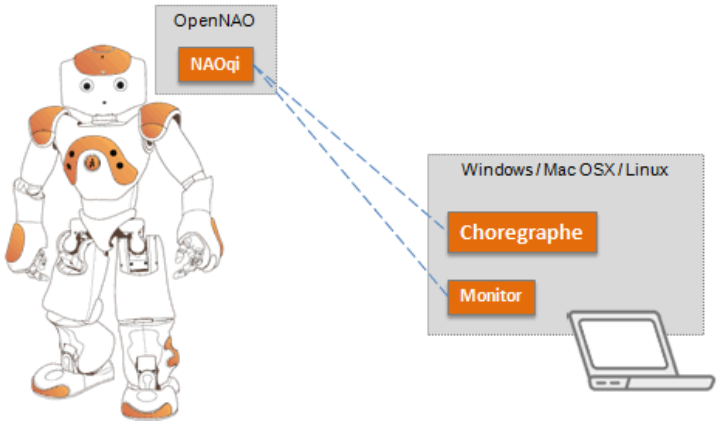


Figure 3.1: NAO and NAOqi system [22]

3.1 Introduction

The integration of different hardware APIs, logic engines, and softwares for animation is necessary in the development of dynamic behaviour modelling frameworks for Socially Assistive Robots like the NAO Robot. Having sufficient knowledge and understanding of the theory and technological tools used. This chapter provides an overview of essential technologies, methodologies and software environments that make up the foundation of the newly developed architecture.

3.2 NAO Motion Control

3.2.1 ALMotion

ALMotion, a core module within the NAOqi OS framework, manages the NAO Robot's physical movements [6]. Thanks to the ALMotion developers can control the robot's actuators using the advanced API that ALMotion offers. Handling the physics such as balance, stiffness of motors and collision detection, enabling external Python scripts to send sets of positional commands straight to the hardware, all this is done by the ALMotion.

3.2.2 Joint-Based Motion and Keyframes

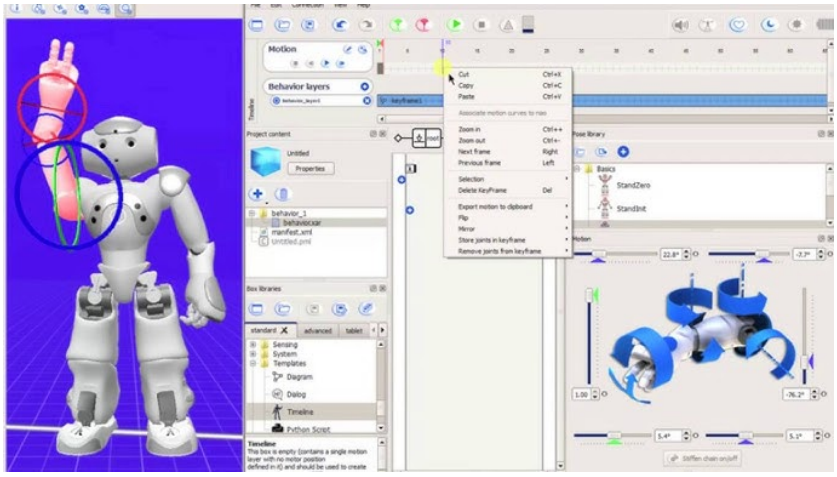


Figure 3.2: NAO Keyframes and timestamps [23]

The structure features about 25 Degrees of Freedom controlled by different motors (e.g., shoulder pitch, elbow roll, head yaw). The motion is developed by making use of keyframes. A keyframe captures a snapshot of the desired angle and position for a specific joint at a precise timestamp. To obtain a complex coordinated routine, multiple keyframes of different joints or limbs are connected together [6].

3.2.3 Bezier Interpolation

To prevent bumpy robotic movements between these keyframes, Bezier curve interpolation is implemented. This is the implementation of mathematical

curves, so instead of seeing mechanical movement which seems very unnatural, and similar to start and stop movements that lack complexity and fluidity, there will be movements that are smooth, featuring the ability to change speed while in motion. This is crucial for achieving human-like movements.

3.3 Gesture Development with Choregraphe

3.3.1 Animation Timeline

For programming the NAO robot a graphical IDE called 'Choregraphe' is used [7]. Choreography is equipped with an Animation Timeline, almost similar to video editing tools. Developers can make use of keyframes where they can save the position or the angles of the limbs and then combine them to produce complex animations.

3.3.2 Virtual Prototyping

Thanks to the virtual 3D environment that the developers can use to virtually prototype movements in Choregraphe. There is a virtual NAO robot that developers, researchers and students use to simulate the movements and observe how the limbs react. This tool is very effective because it gives a picture of how the physical NAO robot will behave before the actual implementation. By the time the animations are implemented into the physical NAO robot, they will be seamless movement depending on how well the developers programmed it.

3.3.3 Kinesthetic Teaching

In cases where the limbs make physical contact with the robot's own body, virtual prototyping might not be sufficient and safe enough. The use of kinesthetic teaching is the most effective way in this case. Kinesthetic teaching is a method where you put the robot in zero-stiffness mode and place the limbs in the desired positions, after that you record and save the angles in the keyframes.

3.4 Speech Synthesis and Audio Processing

For human-robot interaction to be effective the quality and the clarity of the spoken language must be excellent. Improving the quality of the speech is also an area of importance in the development of SARs.



Figure 3.3: Speech processing [24]

3.4.1 Text-to-Speech

Text-to-Speech (TTS) is a technology that turns written word into spoken audio, in other words it transforms text strings into speech. The backend logic systems generate the dialogue dynamically instead of using pre-recorded audio files.

3.4.2 Edge-TTS

Edge-TTS is a Python library that provides access to Microsoft Edge's online text-to-speech service [9]. It uses deep learning models to produce humanlike or natural sounding voices, It analyses hundreds of humanlike, neural voices in different languages and accents. This is crucial for HRI for a therapeutic engagement.

3.4.3 Audio Playback on NAO

Via network streams or audio files the audio generated by the external TTS engines is routed through the NAO robots internal hardware. The NAOqi ALAudioDevice module deals with the playback of the streams using the NAO robot's speakers, controlling the volume and the sound output during interactions. Audio formats such as WAV or MP3 are sent over the network.

3.5 Emotion Recognition

Emotion recognition is when the system uses the cameras for facial-recognition to detect the user's emotions or mode. The use of Facial Expression Recognition (FER) is the core part of this area. Evaluating and analyzing the video frames captured by the cameras is done by different machine learning algorithms that map muscle movements and deformations on the user's face to determine the kind of emotion the user is expressing. Emotions such as happiness, sadness and frustration are detected this way. When the system's reasoning logic receives the variables and inputs of these emotions it gives the appropriate response through the NAO robot [10].

3.6 Answer Set Programming (ASP)

As stated in earlier chapters the logic of the robot's decision making is handled and governed by the Answer Set Programming.

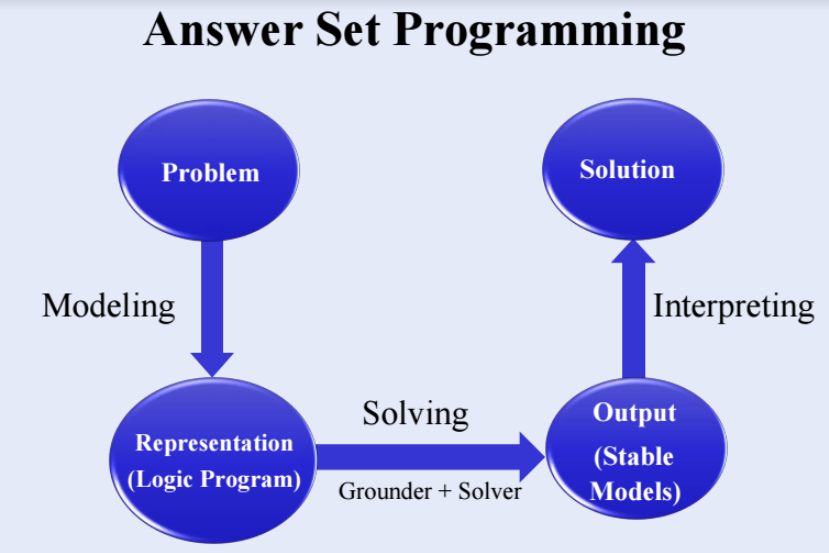


Figure 3.4: ASP model [25]

3.6.1 ASP Fundamentals

ASP is a declarative programming approach, meaning it involves describing what the problem is, which is different from imperative programming e.g. Python, where developers can explicitly write step by step instructions on

how to resolve an issue. ASP makes use of facts, constraints and rules [8]. An assigned logic solver processes this information and knowledge to independently work out the answer sets. An answer is a set of logical facts that represents a valid solution to a given logic program.

3.6.2 Behaviour and Decision Logic

In relation to this thesis, ASP is used to simulate the session with the patients. Variables such as the patient's emotions, the selected option/choice of the exercise and previous performance are given into the logic solver as facts. The ASP rules govern how these facts are used and it enables the system to compute the most logical actions to take. This makes sure that all the choices or steps taken are well within the context and responses make sense.

3.6.3 feedback.asp

In the University of Parma's project architecture, some specific logical fields are separated into dedicated ASP files. The feedback.asp covers the specific rules handling how the system behaves to the input of the patient. The sensory tools provide facts which are then evaluated and put in comparison with the rules defined in feedback.asp, the logic solver then determines the feedback and response that the robot should give in order to assist the patient. In the feedback.asp file it contains all the responses that the NAO gives back.

3.7 Multithreading and Synchronization

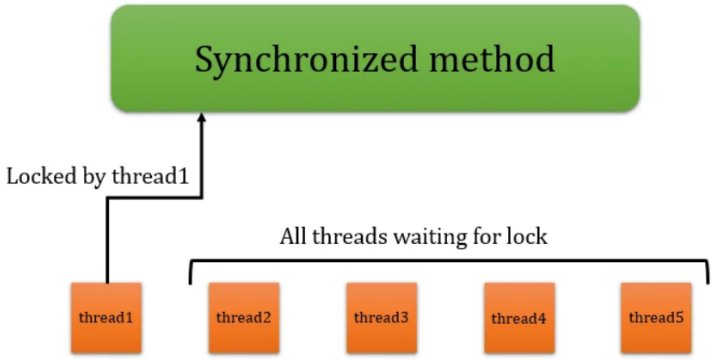


Figure 3.5: Threading illustration [26]

Robotic applications run concurrently. The NAO robot listens to microphones, collects camera stream data, maintains the NAO's stability, and computes future movements simultaneously. Multithreading is a computer programming technique that allows a program to split itself into smaller parts called threads to execute concurrently. Multithreading creates some complex issues, such as race conditions and thread blocking [11].

Synchronization is crucial to the development of advanced human-robot interactions and behaviours. If some threads whether there are audio threads or motion execution are set off without proper synchronization mechanisms, alignment will be lost. Different methods such as single fire callback locks, non-blocking asynchronous calls and strict thread management are necessary to enforce synchronization within the system and maintain a distinction between the processes. These processes will be running in the background and they reflect on the NAO, for example when the NAO robot is speaking while moving its limbs in perfect harmony.

3.8 Chapter Summary

This chapter explained the different technologies used to interface with and manage the NAO Robot. The motor-driven foundation for the movement is provided by the ALMotion and Bezier interpolation. The movements and gestures that give the robot more expressiveness and human-like mannerisms are provided by Choregraphe and kinesthetic teaching. Edge Text-to-Speech addresses the need for high quality audio or speech, and the ASP handles the logical decision making tasks. In addition the rules of multithreading emphasise the processing challenges of carrying out the mentioned features simultaneously. Being guided by these foundations, the next chapter will detail the specific implementations, custom interaction developments and strategies used for synchronisation developed as the main contribution of this thesis.

Chapter 4

Dynamic behaviour modelling

4.1 Introduction

This chapter details the development and implementation of the work done to enhance the NAO robot's capabilities for cognitive rehabilitation systems. The dynamic behaviour modelling framework builds upon the already existing software architecture, the technologies described in the past chapters and emphasises on dealing with the limitations identified in the inherited system especially those concerning expressive gestures, movements, synchronization and reactions to emotions detected.

The implementation is split into multiple interconnected components. Firstly custom gestures were developed and added in a library to expand its collection and give developers and researchers a variety of options. Second was an implementation of a multimodal synchronization mechanism to coordinate speech and motor movement. It addresses inaccurate timing and execution conflicts that happen during an interaction.

Third, the framework was further extended to recognise and react accordingly to dynamic emotional behaviour. Facial emotion recognition was utilised to provide more context on the patient's state to select appropriate responses. A behavioural sparsity rule was introduced to avoid repetition of gestures. Finally the already existing Answer Set Programming (ASP) logic was expanded to integrate physical behaviour alongside dialogue states, making sure the robot's actions are dynamically selected based on the current interaction context.

The following sections explain the development, design decisions, implementation methods and the incorporations of the components, then an overview of the results.

4.2 Custom Gesture Engineering

4.2.1 Gesture Design Requirements

The robot requires physical movements and gestures that are specifically made for a cognitive rehabilitation setting to be of any use for therapeutic purposes. The design requirements dictated that the gestures and movements be highly expressive, recognizable and totally safe for the robot's hardware.

4.2.2 Choregraphe-Based Gesture Development

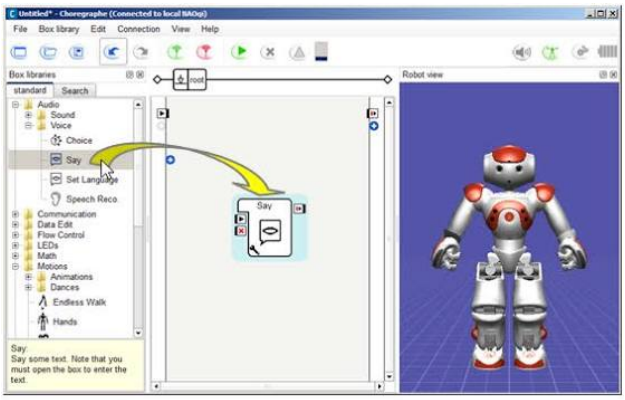


Figure 4.1: Choregraphe interface [27]

Development of gestures makes use of the Choregraphe virtual environment. Open air limb movements such as the breathing gesture, raising arms, pointing, welcoming gesture etc were animated on the platform's timeline editor. It makes use of storing angles and positions in keyframes, then linking them up.

4.2.3 Kinesthetic Teaching

There were some poses which needed to make physical contact, virtual prototyping was not enough because of its risk of limb collision and its inaccuracies. To resolve this issue, the best method was kinesthetic teaching. For example, the creation of the Akimbo was achieved by turning off the stiffness of the motors and placing the hands on its hips, then recording the accurate joint angles.

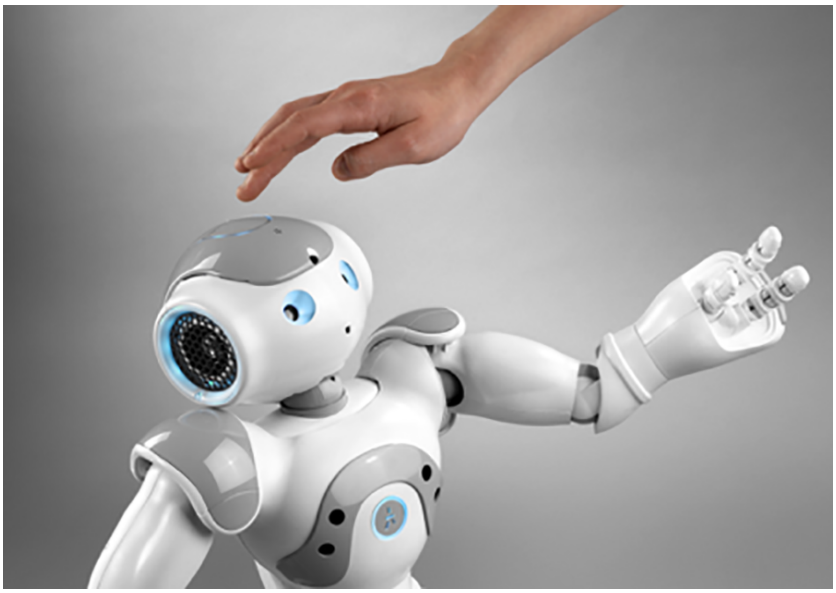


Figure 4.2: NAO being handled [28]

4.2.4 Bezier Keyframe Export and Python Integration

To retrieve these animations from Choregraphe, the motion sequences are exported. It is exported in Bezier curve interpolation for smooth movement, smooth change of speed when the NAO is moving. These exports were then integrated into the backend architecture in python.

4.2.5 Gesture Execution through ALMotion

After converting the motion into Python code, then executing using the ALMotion API. This enables the backend system to activate complex, standalone physical animations dynamically. This does not rely on the visual Choregraphe environment.

4.3 Multimodal Speech–Motor Synchronization

4.3.1 Existing Synchronization Problem

The biggest limitation to the interaction was the coordination between the execution of speech and motion. It could be because of the time it takes to generate the Text-to-Speech (TTS) audio or the time it takes for the motors

to start moving, one could take longer time than the other. Triggering both at the same time resulted in the audio finishing first before the physical gestures were incorporated.

4.3.2 Audio–Motor Latency

Closely examining the latency of the system revealed a consistent delay of the release of the motor command and the actual physical movement of the actuators. This latency interval caused an incoherent user experience, making the NAO robot ineffective in achieving a suitable SAR for engaging patients.

4.3.3 Thread Blocking and Pre-emption

To resolve this issue, a strategy for multithreading synchronization was developed. Thread blocking tools were used to pre-empt the audio thread first, making sure that the TTS stream could not initiate audio output until the thread that controls the motors confirmed that the initial keyframes of a movement or gesture were actively running.

4.3.4 Hardware Spin-up Buffer

```
#!/usr/bin/env python3

import edge_tts
import shutil
import subprocess
import threading
import os
from playsound import playsound
import re
import asyncio
import paramiko
import datetime
import time

# --- MY INTERNSHIP FIX ---
MOTOR_DELAY_SECONDS = 0.4  # Tweak this (0.3 to 0.7) to match the NAO's physical lag
# -------------------------------------
```

Figure 4.3: A 0.4s motor delay, that can be tweaked

A custom hardware spin-up buffer was implemented in the **khule_text2speech_edge_tts.py** module. A constant, **MOTOR_DELAY_SECONDS = 0.4**, was implemented alongside **time.sleep()** blocks. Swapping the callback (a function passed as an argument to another function) execution order, The motors were given a 0.4 second head start to put them in sync with the audio.

```
if self.play_local:
    self.edge_tts_synthesize(text, voice, out_path, rate=self.get_rate(), pitch=self.get_pitch())
    dur = _wav_duration_seconds(out_path)
    if callback_start_speech is not None:
        threading.Thread(target=callback_start_speech, daemon=True).start()

        # --- ADD THIS DELAY ---
        time.sleep(MOTOR_DELAY_SECONDS)
        # ----------------------

    if dur > 0.0:
        play_wav_background(out_path)
else:
```

Figure 4.4: Adding the delay

```
async def _pump_raw():
    nonlocal sent
    async for data in _gen_raw():
        sent += len(data)
        remote_stdin.channel.sendall(data)
    try:
        remote_stdin.channel.shutdown_write()
    except Exception:
        pass

# --- CHANGE ORDER TO GIVE MOTORS A HEAD START ---
if callable(callback_start_speech):
    threading.Thread(target=callback_start_speech, daemon=True).start()
    time.sleep(MOTOR_DELAY_SECONDS)  #ADDED TIME DELAY


self.run_coro([_pump_raw(), ])
# ------------------------------------------------
dur = sent / (24000 * 2 * 1)
return "stream_raw", max(0.05, float(dur))
```

Figure 4.5: Changed order to give the motors a head start

```
            break
        sent += len(data)
        remote_stdin.channel.sendall(data)
    try:
        remote_stdin.channel.shutdown_write()
    except Exception:
        pass

# --- CHANGED ORDER TO GIVE MOTORS A HEAD START ---
if callable(callback_start_speech):
    threading.Thread(target=callback_start_speech, daemon=True).start()
    time.sleep(MOTOR_DELAY_SECONDS)

self.run_coro([_producer_mp3(ff_is_ready), _consumer_pcm(ff_is_ready), ff_task(ff_is_ready)])
# ------------------------------------------------
dur = sent / (24000 * 2 * 1)
return "stream_mp3", max(0.05, float(dur))
```

Figure 4.6: Changed order to give the motors a head start

```
    base = f"nao_{_sanitize(voice)}_{stamp}.wav"
    remote_path = posixpath.join(self.nao_path, base)
    self._scp_put_fallback(out_path, remote_path)

    # --- SWAPPED THE ORDER HERE, PUT THE CALLBACK BEFORE THE REMOTE PLAY --
    try:
        if callable(callback_start_speech):
            threading.Thread(target=callback_start_speech, daemon=True).start()
    except:
        print("ERROR CALLBACK MOVEMENT")

    self._remote_play_wav(remote_path)
    # ----------------------------

    return remote_path, dur
```

Figure 4.7: Swapped the order and put the callback before the remote play

4.3.5 Motor Command Collision

With multithreading there will exist risks of command collision, especially in relation to the robot's resting state. Some functions that prematurely called the rest_position were deleted from the active gesture pipeline, making sure that ongoing movements or animations were not cut short.

4.3.6 Callback Management and Edge-TTS

For managing the asynchronousness of the TTS, single-fire callback locks were implemented. The locks make sure that the fallback TTS streams or interruptions did not cause the pipeline to crash by triggering duplicated motion commands, ensuring coherence and stability during interactions.



Figure 4.8: Single-fire callback.

4.3.7 Final Synchronization Pipeline

The **say_with_move()** function is the final result of these solutions. It's a unified pipeline that integrates both the audio stream and the delayed ALMotion execution into a single block that is thread safe.

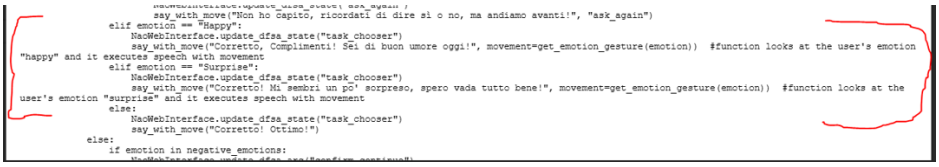


Figure 4.9: Part in say_with_me block

4.4 Dynamic Emotion-Driven Behaviour

4.4.1 Dynamic Emotion-Driven Behaviour

In addition to scripted gestures, there was an upgrade which gave the NAO robot the ability to react autonomously to the user's mood. Using the computer vision on the NAO robot, the logged emotional data was put into use and redirected to dynamically affect the robot's physical behaviour.

4.4.2 Dynamic Helper Functions

```
# ========================================
# MY NEW HELPER FUNCTION THAT AUTOMATES GESTURES BASED ON EMOTIONS DETECTED ON CAMERA:
def get_emotion_gesture(emotion_detected):
    if emotion_detected == "Happy":
        return "both_arms_hello"
    elif emotion_detected in ["Sad", "Fear", "Anger"]:
        return "no_sign_right_arm"
    elif emotion_detected == "Surprise":
        return "raise_arms"
    else:
        return "gesticulate" # Default talking movement
# ========================================
```

Figure 4.10: Helper function that automates gestures based on detected emotions

This helper function **get_emotion_gesture(emotion_detected)** was developed in the Python backend. The function computes the user's emotion in real time and returns a gesture that behaves accordingly.

4.4.3 Context-Aware Physical Responses

The reaction is context aware. For example if the computer vision and all the related parts of the system detect surprise on the user's face, the helper function returns the "raise_arms" gesture and if happiness is detected it returns the "both_arm_hello" gesture. This makes the NAO robot empathetic.

4.4.4 Behavioural Sparsity and the 30/70 Rule

To avoid repetition and to avoid predictability, a probability framework called the "30/70 Sparsity Rule" is implemented. So instead of reacting to all the emotions detected, the logic engine activates a more noticeable response 30% of the time and a minor gesture 70% of the time. The sparsity rule maintains a balance between the robot's mannerisms and tries to make it human-like.

```
# --- ADDED: Use my script if ASP does not give directions! ---
elif emotion_recognition == 1 and last_user_emotion:
    # 30% chance to do a large, emotion-based gesture
    if random.random() < 0.30:
        print("Emozione non ricevuta da feedback, la computo da emozione: " + last_user_emotion)
        movement = get_emotion_gesture(last_user_emotion)
    else:
        # 70% chance to do a slight, natural hand movement or head nod instead of freezing!
        movement = random.choice(["speech_right_arm", "speech_left_arm", "nod"])
# ------------------------------------------------------------------
```

Figure 4.11: Sparsity rule

4.5 ASP Behaviour Integration

4.5.1 Existing Behaviour Logic

Updating the core decision-making engine was the final phase of the implementation. The already existing Answer Set Programming (ASP) logic was limited to mapping the dialogue based on the results or the responses of the users in the games or exercises.

4.5.2 Extension of feedback.asp

```
% --- GESTURES FOR EXPLAINING RULES ---
feedback(introduce_s,motion,"gesticulate").    %to make the robot more engaging by u
feedback(introduce_r,motion,"gesticulate").
feedback(introduce_d,motion,"gesticulate").
feedback(introduce_c,motion,"gesticulate").    % till here

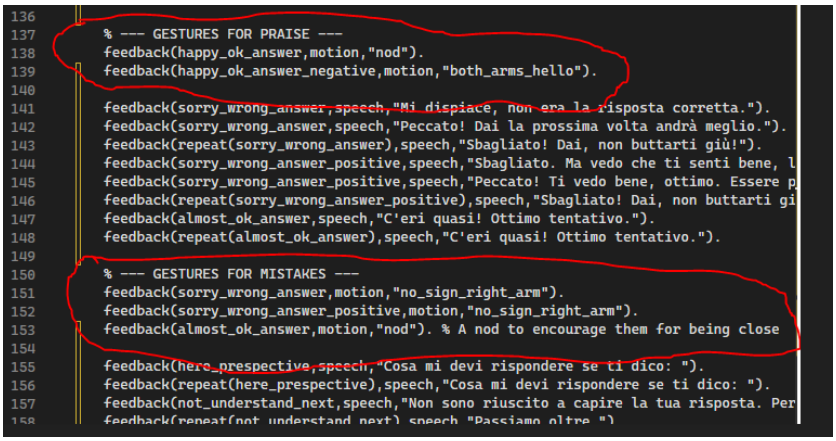
feedback(begin_task,speech,"Bene, cominciamo.").
feedback(begin_task,speech,"Si comincia!").
feedback(repeat(begin_task),speech,"Cominciamo!").
feedback(wait_begin_task,speech,"Andrà tutto bene vedrai! Cominciamo!").
```

Figure 4.12: Part of feedback.asp extension

The declarative logic within the feedback.asp file was significantly broadened and updated to include gestures. These changes merged the speech and movement planning into a single united reasoning process.

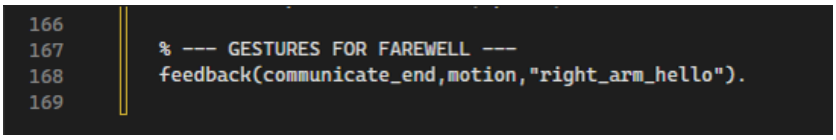
4.5.3 Behaviour Facts

Specific gestures or reactions are mapped to specific answers and inputs. This is mostly linked to therapeutic context. “Gesticulate” gesture which is linked to explanations of rules, games and exercises. “Nod” and ”both_arms_hello” were assigned as actions of praise. ”no_sign_right_arm” to indicate error.



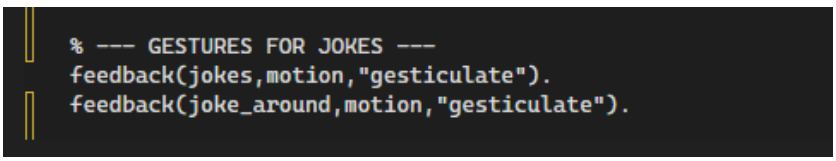
```
136
137     % --- GESTURES FOR PRAISE ---
138     feedback(happy_ok_answer,motion,"nod").
139     feedback(happy_ok_answer_negative,motion,"both_arms_hello").
140
141     feedback(sorry_wrong_answer,speech,"Mi dispiace, non era la risposta corretta.").
142     feedback(sorry_wrong_answer,speech,"Peccato! Dai la prossima volta andrà meglio.").
143     feedback(repeat(sorry_wrong_answer),speech,"Sbagliato! Dai, non buttarti giù!").
144     feedback(sorry_wrong_answer_positive,speech,"Sbagliato. Ma vedo che ti senti bene, l
145     feedback(sorry_wrong_answer_positive,speech,"Peccato! Ti vedo bene, ottimo. Essere p
146     feedback(repeat(sorry_wrong_answer_positive),speech,"Sbagliato! Dai, non buttarti gi
147     feedback(almost_ok_answer,speech,"C'eri quasi! Ottimo tentativo.").
148     feedback(repeat(almost_ok_answer),speech,"C'eri quasi! Ottimo tentativo.").
149
150     % --- GESTURES FOR MISTAKES ---
151     feedback(sorry_wrong_answer,motion,"no_sign_right_arm").
152     feedback(sorry_wrong_answer_positive,motion,"no_sign_right_arm").
153     feedback(almost_ok_answer,motion,"nod"). % A nod to encourage them for being close
154
155     feedback(here_prespective,speech,"Cosa mi devi rispondere se ti dico: ").
156     feedback(repeat(here_prespective),speech,"Cosa mi devi rispondere se ti dico: ").
157     feedback(not_understand_next,speech,"Non sono riuscito a capire la tua risposta. Per
158     feedback(repeat(not_understand_next),speech,"Passiamo oltre.").
```

Figure 4.13: Gestures mapped to specific behaves



```
166
167     % --- GESTURES FOR FAREWELL ---
168     feedback(communicate_end,motion,"right_arm_hello").
169
```

Figure 4.14: Feedback expansion



```
% --- GESTURES FOR JOKES ---
feedback(jokes,motion,"gesticulate").
feedback(joke_around,motion,"gesticulate").
```

Figure 4.15: Feedback expansion

4.5.4 Integration with Dialogue States

```
def reason():
    """Logica del ragionatore con gestione delle risposte."""
    end = False

    while NaoWebInterface.conversation_state == 'pause':
        time.sleep(0.1)

    NaoWebInterface.update_dfsa_state("welcome_ask_name")
    say_with_move("Ciao. Come ti chiami?", movement="both_arms_hello")
    name, _, _ = nao_listen.record("Come ti chiami?")
    name = getinfo.get_name(name)

    # Invia informazioni sull'arco uscente - solo l'etichetta
    NaoWebInterface.update_dfsa_arc("name_response")

    NaoWebInterface.update_dfsa_state("is_name_right")
    say_with_move(f"Ciao {name}. è un piacere conoscerti!", movement="nod")

    # Invia informazioni sull'arco uscente - solo l'etichetta
    NaoWebInterface.update_dfsa_arc("confirmation")

    NaoWebInterface.update_dfsa_state("memory_exercise")
    say_with_move(f"Voglio farti alcune domande per allenare la tua memoria.", movement="gesticulate")
```

Figure 4.16: Updated say_with_me

Integrating the updated say_with_move() function into the core reason() loop was done to execute the changes in the previous sections. This enabled both the speech and the gesture to be executed simultaneously.

4.5.5 Dynamic Behaviour Selection

```
try:
    excel_logger.log_interaction(
        game="memory_exercise",     # <--- ADDED THESE dummy variables for game and difficulty inside the reason() function.
        difficulty=1,               # <--- Because reason was crashing due to not having these arguments
        emotion_before="",
        question=item["question"],
        time_elapsed=resp_time,
        user_answer=answer,
        emotion_after=emotion if emotion_recognition == 1 else "",
        is_correct=is_correct,
        expected_answer=item["correct_answer"],
        gaze_result=gaze_flag
    )
except Exception as e:
    print(f"[EXCEL] Errore log_interaction: {e}")
```

Figure 4.17: Reason was crashing because of missing variables

During the integration the reason was crashing because of system stability issues that developed some variables and values were missing. To prevent this crashing during dynamic execution, dummy variables such as game=”memory_exercise” and difficulty=1 were inserted in the execution block above. This ensures consistent and persistent data tracking.

4.6 Overall System Architecture

The incorporation of all these components build a largely robust, multimodal system. The diagram below illustrates architecture of the system.

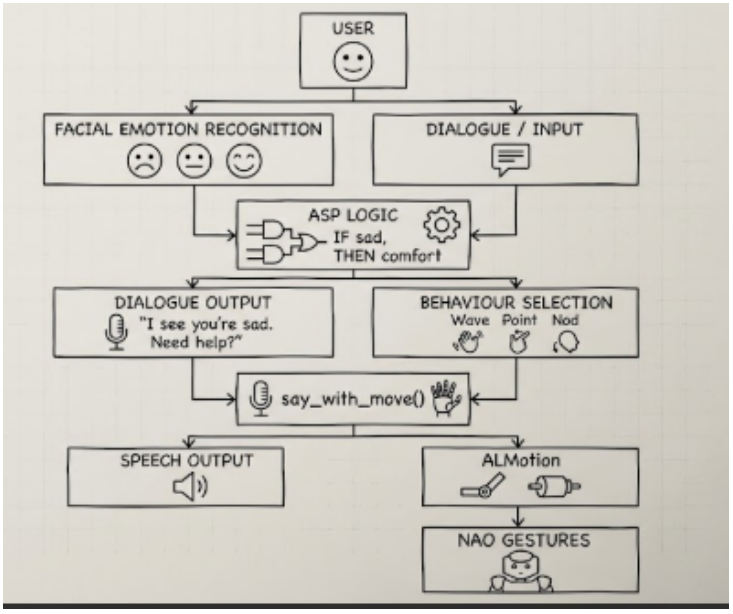


Figure 4.18: Overview of the system architecture

4.7 Chapter Summary

This chapter explains the comprehensive engineering modifications applied to improve the NAO robot from lacking expressiveness to a dynamic social companion. By developing custom gestures, resolving synchronisation, fixing audio-motor latency issues, introducing the sparsity rule, broadening the ASP logic, there was a complete improvement in the behaviour of the NAO robot. The following chapter gives a summary of the validation of the experiment carried out.

Chapter 5

Validation and Evaluation

5.1 Introduction

Validating the system's performance after the new implementations was an important part of the process. It is a way to understand whether the implementations were successful or not. The chapter outlines the methods used for testing to analyse the behavioural framework. The focus was on verifying if the execution was safe, the precision of the synchronization, and how well the integrated logic on the physical hardware is functioning.

5.2 Experimental Setup

5.2.1 NAO Hardware Configuration

All physical hardware testing was carried out in the Robotics and Intelligent Machines Laboratory (RIMLab) at the University of Parma. The trials during the experiment used the physical NAO humanoid robot to evaluate the mechanical inertia (inertia is the natural tendency of an object to resist any change in its state of motion or rest), limb movements and how the hardware and software are communicating.

5.2.2 Software Environment

The evaluation environment linked the development that was made and the physical implementation. Initially the virtual prototyping was executed on the Choregraphe platform. After that the compiled code and gesture libraries were sent over to the RIMLab for live testing on the NAO platform, checking the log on the dashboard for any errors.

5.2.3 Test Scenarios

Testing was done iteratively throughout the development process. During the initial test sessions, several executions did not produce the desired behaviour. In particular, the NAO robot did not always perform the gestures as expected, and the speech and motor actions were not adequately synchronized. These initial failures provided the basis for identifying and addressing issues in gesture execution, synchronization, and the interaction between the different system components.

A range of physical behaviours was tested, including breathing, welcoming, expressing disagreement through the "no way" gesture, stomach ache, headache, calming gestures, fist salutes, and bowing. Testing these different behaviours allowed the gesture execution and the corresponding physical responses to be observed across a variety of interaction contexts.

The testing process was divided into two main stages. The first stage involved testing individual gestures in the virtual environment to verify their movement sequences and assess their safety before execution on the physical NAO robot. This stage allowed potential problems with the positioning and movement of the robot to be identified before transferring the gestures to the physical platform.

The second stage involved testing the complete scripts in which the ASP decision-making logic and speech–motor synchronization mechanisms were executed together. These tests were carried out in the RIMLab, where the behaviour of the physical NAO robot could be observed during simulated interaction scenarios. Testing was carried out with the assistance of a senior student, who assisted in observing the sessions, identifying bugs, and evaluating whether the robot's responses appeared natural and appropriately synchronized.

The iterative nature of the testing process allowed problems identified during early executions to be addressed and then retested. The final evaluation therefore focused not only on whether individual gestures could be done successfully, but also on whether the different components of the system could operate together as a coherent interaction pipeline.

5.3 Gesture Validation

5.3.1 Hardware Safety Protocol and Instability

It was highly recommended to enforce safety measures during the execution of the tests on the physical robot. Observations revealed that naturally the NAO is not very stable, it always felt like it was about to fall over. The

NAO often tilted forward and back. The tilt was minor but revealed that the NAO robot lost its center of gravity. To avoid balance failure the custom gesture library was mainly focused on developing the upper-body movements. During the tests a safety precaution of setting a closed perimeter using the developer's hands around the NAO robot was taken to protect the robot from falling over.

5.3.2 Validation of Kinesthetic Teaching

For gestures or movements requiring physical contacts, due to the limitations such as collision risks and inaccuracies in angles and positions of solely virtual prototyping, kinesthetic teaching had to be implemented as a necessity. It was only necessary for gestures like the Akimbo and others that were of a higher degree of difficulty in the virtual environment. Kinesthetic teaching solved this problem and was completely safe.

5.3.3 Gesture Library Deployment

Once everything was in order, safety was ensured and the motion animations were running successfully. These movements and gestures were saved in a custom library, allowing users to use that library at any given time or work on that project that contains all the gestures.

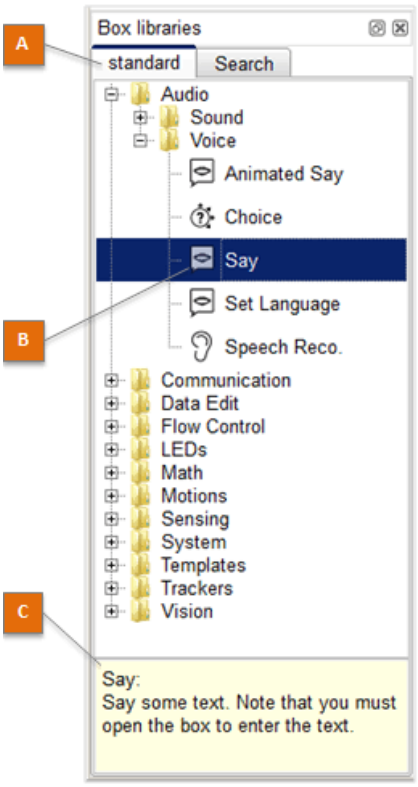


Figure 5.1: Box libraries panel [29]

5.4 Multimodal Synchronization Evaluation

5.4.1 Audio–Motor Timing

The process of evaluating the synchronization took place, the main components of that process were the dialogue generation and the movement of the hardware. The testing confirmed that the motor delay was successful and resolved the latency gap. This was evident by how the audio spoken by the NAO robot was aligning with the physical movements.

5.4.2 Thread and Callback Stability

The single-fire callback locks showed they were effective during the testing. All the unintended termination and the duplicate command triggering was handled by this single-fire callback. The system gained fluidity.

5.5 Emotion-Driven Behaviour Evaluation

5.5.1 Offline Simulation and Logic Verification

Before the incorporation with the hardware or while working from home, a testing block that includes visual indicators such as rocket emojis confirmed that the ASP engine was correctly mapping the emotion states to the custom helper function.

5.5.2 Behavioural Sparsity Evaluation

When integrated to the hardware, the sparsity rule that was implemented effectively controlled the NAO robot's expressiveness. Observing the way the robot was behaving confirmed that the robot became more empathetic and human-like mannerisms were clearly visible. This maintained a balance and avoided any sort of exaggeration from the NAO robot.

5.6 Integrated System Validation

5.6.1 Live Code Execution and Log Monitoring

The last stage of the evaluation validated the backend, specifically the feedback.asp modifications and the say_with_move updates. The main tool was the log data from the NAO Dashboard which was used to monitor system logs, track ASP logic states, and monitor the flow of data.

5.6.2 Iterative Debugging and Error Resolution

Some software bugs were identified, including the case where the logic engine was able to register commands like gesticulate but the NAO robot was unable to execute the corresponding physical movements. Because of the careful analysis that was carried out alongside the senior student, the issues were identified and the modifications were made. After the changes, the code was now stable and there was now proper implementation. The robot was successfully performing the gesture.

5.6.3 System Stability and Excel Logging

Also detected was an issue regarding the external data logging. The Excel logger was crashing when interacting with the dynamic execution loops of the new modifications. The use of dummy variables namely "game=memory_exercise" and "difficulty=1" into the execution block, helped prevent this crash from happening again and enabled the continuous tracking of data.

5.7 Discussion

Through the validating of the experiments and project there was clear confirmation that the objectives of this thesis were successfully met. Integration of the custom gestures, audio-motor synchronisation, and the ASP driven reaction to emotion on the physical NAO robot was smooth and the components all worked cohesively. Kinesthetics teaching was the perfect escape of the structural limitation obstacle. Use of NAO dashboard logs aided in making sure all bugs were eliminated. All these processes produce an active and suitable socially assistive NAO robot.

5.8 Chapter Summary

This chapter detailed the testing of the dynamic behavioural framework that was carried out at the University of Parma's RIMLab. A combination of hardware safety protocols, debugging, observation, demonstrations, reviewing the performance and other checks were carried out to identify if any errors exist. Successful tests identified a lot of errors which resulted in the solutions mentioned in the previous sections.

The following chapter will conclude the thesis, summarizing the overall contributions and proposing avenues for future research.

Chapter 6

Conclusion and Future Work

6.1 Introduction

This final chapter concludes the thesis by summarizing the research and developmental efforts carried out during the internship at the University of Parma. It offers a thorough overview of the developed dynamic behavioural framework, explicitly, it explains the main technical contributions to the pre-existing cognitive rehabilitation system and it notices the current drawbacks of the limitations. Finally it proposes potential paths for future research and development to improve human-robot interaction in therapeutic environments.

6.2 Summary of the Work

The thesis is aimed at improving the interactions between humans and Socially Assistive Robots used in cognitive rehabilitation. Chapter 1 established the clinical necessity for cognitive rehabilitation and introduced The NAO humanoid robot as a therapeutic instrument. Chapter 2 reviews the already existing rehabilitation project done at the University of Parma by researchers and students, identifying critical limitations in its majorly static interactions and lack of physical expressions. Chapter 3 detailed the core technologies and methodologies, including ALMotion API, Choregraphe and the Answer Set Programming. Chapter 4 presented the core implementation explaining the creation of the modifications that were carried out including synchronized speech and physical movements, in addition the integration of reactions that are based on the user's emotions Finally Chapter 5 validated the research and development on the NAO robot, evaluating the improvements that were carried out, improvements such as stability, safety and expressive behaviours

for a human-robot interaction.

6.3 Main Contributions

The main objective of this project was to transform the NAO robot from a non-expressive robot to a dynamic and expressive SAR. To achieve this, the following main contributions were implemented:

- **Custom gesture development:** Designing and creation of a custom collision safe gesture library for cognitive therapy and interactions.
- **Python/ALMotion integration:** The export of animations from Choregraphe into Python execution arrays.
- **Synchronization of the speech and limbs:** ensuring physical gestures align with the robot's speech or audio.
- **Management of threads:** The implementation of blocking tools and single-fire function to avoid collision of commands and stability of the system.
- **Emotion based reaction:** The development of helper functions that map real time facial emotion recognition to contextualise the physical reactions.
- **30/70 Sparsity rule:** The introduction of a sparsity rule to control emotional reactivity, preventing exaggeration and maintaining human-like mannerisms.
- **ASP Behaviour Integration:** The expansion of the declarative logic engine to smoothly merge speech generation and physical action mapping into a unified decision making process.
- **Physical NAO Validation:** The testing that was carried out that confirmed success or failure and then the debugging that came after.

6.4 Limitations

Even after successful developments and implementation, there were still some limitations that remain:

- Testing did not cover all aspects of cognitive therapy.

- The system has not been given an extensive amount of time in a large scale environment like a hospital.
- The NAO possesses questionable stability, its problem of tilting still remains.
- The emotion recognition relies heavily on the quality of the computer vision, misclassifications can happen.
- Network quality could impact the speech to physical movement synchronisation, because Edge-TTS is involved.
- The mapping of the physical actions to emotions is generalised, for the best therapeutic behaviour it all depends with the specific patient and need personalisation.

6.5 Future Work

The work done in the internship provided a solid foundation for more improvements and left some areas that can be improved such as:

- Increasing the set of emotions, some emotions have still not been implemented.
- Personalising the gestures depending on the patient because humans are different and can have different reactions to different things, so knowing the specific patient in particular will return more reliable data.
- Mechanisms that will allow the robot to learn patients behaviour and energy then adjust the 30/70 sparsity rule. That way the robot can return the same energy the patient gives. Think of a robot chatting to a super excited kid.
- Making the robot learn and know its patient for future purposes that way the robot does not have to start from scratch the next time it sees the same patient.
- Replacing structured ASP dialogue with a smart Large Language Model, this will increase the fluidity of engagements.
- More working on speech and gesture timing to be closer to human level.
- Full body gestures and making the NAO robot completely autonomous.

6.6 Final Remarks

The integration of multimodal communication is crucial for the advancement of Socially Assistive Robots in healthcare and in all sectors. Successfully coordinating motor movements with speech and emotion based logic, this thesis extended the University of Parma's cognitive rehabilitation platform from a non-expressive interface toward a significantly more synchronized and expressive system. Ultimately, this dynamic behaviour modelling ensures that the NAO robot interacts with patients not only as an automated tool but as a natural, engaging and empathetic therapeutic companion.

Acknowledgements

I would like to take this opportunity to thank everyone who supported me during my studies and the development of this thesis.

First of all I would like to thank my supervisor, **Professor Dal Palù** for his guidance, support and feedback during my internships. Professor's advice and guidance helped me better understand the technical challenges of the project and allowed me to tackle this huge obstacle. I would like to thank the senior students, especially **Leonard Consalvo**, who was helping me throughout my internships and also the co-supervisor **Prof. Riccardo Monica**.

I would also like to thank the **University of Parma** and everyone involved in the project for giving me the opportunity to work on this challenging project. Working with the NAO robot and contributing to an existing work was really exciting and gave me the opportunity to apply what I had learned during my studies to a real world problem, while also learning new things along the way.

I am very thankful to the people who helped me during my internship because they sacrificed their time and effort to help me. The discussions, advice and availability that was shared made this project easier than it would've been without them. Their support made the experience much more enjoyable and doable. Last but not least I would like to thank my **family and friends** for their continued support and encouragement throughout my studies. Completing this thesis would not have been possible without the support of each and every one of them.

Thank you to everyone who was part of this journey.

Bibliography

[1] World Health Organization, "Rehabilitation," World Health Organization, Apr. 22, 2024. https://www.who.int/news-room/fact-sheets/detail/rehabilitation

[2] K. Suresh Kumar, S. Samuelkamaleshkumar, A. Viswanathan, and A. S. Macaden, "Cognitive rehabilitation for adults with traumatic brain injury to improve occupational outcomes," *Cochrane Database Syst. Rev.*, vol. 2017, no. 6, p. CD007935, Jun. 2017. https://pmc.ncbi.nlm.nih.gov/articles/PMC6481568/

[3] A. Tapus, M. J. Matarić, and B. Scassellati, "Socially assistive robotics [Grand challenges of robotics]," *IEEE Robotics & Automation Magazine*, vol. 14, no. 1, pp. 35–42, 2007. DOI: 10.1109/MRA.2007.339605.

[4] M. Zubair, C. Tamantini, R. De Benedictis, F. Fracasso, A. Umbrico, G. Beraldo, A. Orlandini, and G. Cortellessa, "Socially Assistive Robots for Cognitive Assessment and Training in Older Adults: A Systematic Review," *Robotics*, vol. 15, no. 8, p. 152, Aug. 2026. https://doi.org/10.3390/robotics15080152

[5] D. Gouaillier *et al.*, "The NAO humanoid: A combination of performance and affordability," arXiv:0807.3223, 2008.

[6] SoftBank Robotics, NAO Technical Documentation.

[7] E. Pot, J. Monceaux, R. Gelin, and B. Maisonnier, "Choregraphe: A graphical tool for humanoid robot programming," *18th IEEE International Symposium on Robot and Human Interactive Communication*, 2009. DOI: 10.1109/ROMAN.2009.5326209

[8] V. Lifschitz, "What Is Answer Set Programming?" in *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence*, 2008.

[9] rany2, *edge-tts*, GitHub repository.

[10] A. Lopez-Rincon, “Emotion Recognition using Facial Expressions in Children using the NAO Robot,” *2019 International Conference on Electronics, Communications and Computers (CONIELECOMP)*, pp. 146–153, 2019. DOI: 10.1109/CONIELECOMP.2019.8673111.

[11] A. Baksheiev, “Understanding the Fundamentals of Multi-Threading: A Beginner’s Guide,” Medium, Mar. 27, 2025. [Online]. Available: https://medium.com/@anton.baksheiev/understanding-the-fundamentals-of-multi-threading-a-beginners-guide-b6585844a538

[12] Tulsulkar, G., Mishra, N., Thalmann, N. M., Lim, H. E., Lee, M. P., & Cheng, S. K. (2021). Can a humanoid social robot stimulate the interactivity of cognitively impaired elderly? A thorough study based on computer vision methods. *The Visual Computer*, 37, 3019–3038. https://doi.org/10.1007/s00371-021-02242-y

[13] E. Martínez-Martín, F. Escalona, and M. Cazorla, “Socially Assistive Robots for Older Adults and People with Autism: An Overview,” *Electronics*, vol. 9, no. 2, p. 367, 2020. https://doi.org/10.3390/electronics9020367

[14] University of Wisconsin–Madison Robotics Group, ”NAO,” Robotics at the UW-Madison. https://robotics.wisc.edu/robots/nao/

[15] E. Wirbel, S. Bonnabel, A. de La Fortelle, and F. Moutarde, “Humanoid Robot Navigation: Getting Localization Information from Vision,” *Journal of Intelligent Systems*, vol. 23, no. 2, pp. 113–132, 2014. https://doi.org/10.1515/jisys-2013-0079

[16] K. Jayagopi, S. Sheikhi, D. Klotz, et al., “The Vernissage Corpus: A Multimodal Human-Robot-Interaction Dataset,” Idiap Research Report, 2012.

[17] Aldebaran Robotics, “Webots for NAO,” NAO Software 1.14.5 Documentation. [Online]. Available: http://doc.aldebaran.com/1-14/software/webots/webots_index.html

[18] GeeksforGeeks, ”Introduction of Deadlock in Operating System,” GeeksforGeeks, Apr. 15, 2026. [Online]. Available: https://www.geeksforgeeks.org/operating-systems/introduction-of-deadlock-in-operating-system/

[19] UK researchers investigate robots that can keep secrets,” The Telegraph, Feb. 27, 2014. [Online]. Available: https://www.telegraph.co.uk/te

chnology/news/10665140/UK-researchers-investigate-robots-that-can-keep-secrets.html

[20] P. Alberti, "NAO e autismo," Innovation for Education, Dec. 16, 2021. [Online]. Available: https://www.innovationforeducation.it/approfondimento/nao-e-autismo/

[21] Image generated by Gemini using the section as a prompt.

[22] Aldebaran Robotics, "Software in and out of the robot," NAO Software 1.14.5 Documentation. [Online]. Available: http://doc.aldebaran.com/1-14/getting_started/software_in_and_out.html

[23] EWU Robotics, "Creating Timelines for NAO Part 1: Basics," Online Video, Aug 2016. https://youtu.be/ikNX8A-3P98

[24] aiola, "What is Speech Processing," aiola. [Online]. Available: https://aiola.ai/glossary/speech-processing/

[25] Stack Overflow, 'answer-set-programming' tag wiki," Stack Overflow. [Online]. Available: https://stackoverflow.com/tags/answer-set-programming/info

[26] Ö. Memes, "What is Thread Synchronization?," Medium, May 20, 2024. [Online]. Available: https://medium.com/@omermemes83/what-is-thread-synchronization-b8ba58df855b

[27] Aldebaran Robotics, "Hello World 1 - using Choregraphe," NAO Software 1.14.5 Documentation. [Online]. Available: http://doc.aldebaran.com/1-14/getting_started/helloworld_choregraphe.html

[28] Aldebaran's New Nao Robot Demo," IEEE Spectrum, Dec. 21, 2010. [Online]. Available: https://spectrum.ieee.org/aldebaran-new-nao-robot-demo

[29] Aldebaran Robotics, "Box libraries panel," Aldebaran 2.1.4.13 Documentation. [Online]. Available: http://doc.aldebaran.com/2-1/software/choregraphe/panels/box_libraries_panel.html